A Beginner's Practical Guide to Ports and Protocols

Ports and protocols act like digital apartment numbers for internet traffic reaching your server. Understanding them helps you instantly diagnose why applications cannot connect to databases or web services.

What you'll learn:

1. Web Traffic (Ports 80 & 443)
2. Secure Shell Access (Port 22)
3. Database Port Connectivity (Port 5432)
4. Managing Local Dev Ports (Port 3000)

Aman Raj Singh
Cloud Operations Engineer @ iManage
linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

Web Traffic (Ports 80 & 443)

Web traffic travels over port 80 for unencrypted HTTP and port 443 for secure encrypted HTTPS. When you type a website URL into a browser, it automatically attaches to these default ports. Testing these ports using terminal tools lets you verify server availability without opening a web browser.

```
$curl -Iv https://api.github.com
```

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

Secure Shell Access (Port 22)

Port 22 is the default gateway for managing Linux servers remotely using Secure Shell (SSH). It encrypts administrative traffic so passwords and terminal commands remain safe from network snooping. If connections fail with a timeout error, firewall rules on port 22 are usually the root cause.

```
$ssh -v ubuntu@192.168.1.50
```

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

Database Port Connectivity (Port 5432)

PostgreSQL database engines listen for application queries on default port 5432 by default. Applications fail immediately on startup if network paths to this port are blocked or misconfigured. You can quickly verify database port access from your app container using Netcat.

```
$nc -zv postgres-db.example.com 5432
```

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

Managing Local Dev Ports (Port 3000)

Developers frequently encounter port conflict errors when running web apps locally on port 3000 or 8080. This occurs when a previously crashed process is still secretly holding onto the port in the background. You can locate the hung process ID and terminate it safely from your command line.

```
$lsof -i :3000 && kill -9 $(lsof -t -i:3000)
```

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

+ Always check security group and firewall rules first when network requests time out.
+ Use curl -v or nc to test network connectivity before assuming application code is broken.
+ Never expose database ports like 5432 or 3306 directly to the public internet.
+ Document standard local development ports clearly in your project README file.
+ Always enforce HTTPS on port 443 for production web applications to safeguard data.

Watch Out For

! Assuming a backend application is down when a network firewall is just blocking the port.
! Hardcoding target server IP addresses and port numbers directly inside source code files.
! Leaving background processes running locally that permanently occupy development ports.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mramanrajsingh